

# smallgameagent 实验报告

基于 LLM/VLM 与规则的小游戏 Agent

smallgameagent 项目组

2026 年 7 月 20 日

## 目录

|       |                                                        |   |
|-------|--------------------------------------------------------|---|
| 1     | smallgameagent 实验报告（第三轮：分层架构 + 批量框架）                   | 2 |
| 1.1   | 0. 一页结论                                                | 2 |
| 1.2   | 1. 分层多 Agent 架构（实验 F）                                  | 2 |
| 1.2.1 | 设计                                                     | 2 |
| 1.2.2 | 实现                                                     | 3 |
| 1.2.3 | 批量实验结果                                                 | 3 |
| 1.3   | 2. Node.js 高级逻辑移植（实验 H）                                | 3 |
| 1.4   | 3. 批量实验框架（实验 G）                                        | 4 |
| 1.4.1 | 架构                                                     | 4 |
| 1.4.2 | 使用方式                                                   | 4 |
| 1.4.3 | 产出                                                     | 4 |
| 1.4.4 | 轨迹数据格式                                                 | 4 |
| 1.5   | 4. 同学系统对比分析                                            | 5 |
| 1.6   | 5. 多游戏泛化实验（第四轮）                                        | 5 |
| 1.6.1 | 5.1 Generic fallback 让 22 游戏可驱动                        | 5 |
| 1.6.2 | 5.2 Probe 终止假阳性修复                                      | 5 |
| 1.6.3 | 5.3 修正后多游戏结果（5 游戏 × 2 模式）                              | 6 |
| 1.6.4 | 5.4 全游戏自动校准（auto_calibrate.py -all, 22 游戏）             | 6 |
| 1.6.5 | 5.5 Tap-to-Move 驱动策略                                   | 6 |
| 1.7   | 6. 全游戏 × 多模式批量矩阵                                       | 7 |
| 1.7.1 | 6.1 A_full（7 个 joystick 游戏 × 4 模式 × 2 seeds = 48 runs） | 7 |
| 1.7.2 | 6.2 B_tap（15 个 tap-only 游戏 × 2 模式 × 2 seeds = 60 runs） | 7 |
| 1.8   | 7. 云端 API 策略生成 vs 纯 rule                               | 8 |
| 1.9   | 8. VLM 视觉管线                                            | 8 |
| 1.10  | 9. 训练数据生成                                              | 8 |
| 1.11  | 10. L2 输出契约修复                                          | 8 |

|                         |   |
|-------------------------|---|
| 目录                      | 2 |
| 1.12 11. 后续建议 . . . . . | 9 |

# 1 smallgameagent 实验报告（第三轮：分层架构 + 批量框架）

> 2026-07-20。配套：\_\_CODE\_\_0000\_\_（方案）、\_\_CODE\_\_0001\_\_（过程数据）。  
> 本轮重点：分层多 Agent 架构、Node.js 高级逻辑移植、批量实验框架、数据采集管线。

## 1.1 0. 一页结论

- \_\_BOLD\_\_0000\_\_：实现 L0 规则（每步 ~0ms）+ L1 本地 VLM（每 5 步 ~5s）+ L2 云端 API（每 15 步 ~3s）三层决策。批量实验中 hierarchical 模式 composite=0.150，但 L1 因本地 VLM 未启动而退化为 L0+L2；L2 kimi-k2.7-code 的思考链输出导致 JSON 解析失败。\_\_BOLD\_\_0001\_\_：架构可行，但需要 (a) 本地 VLM 常驻 + (b) 更强的 L2 输出契约。
- \_\_BOLD\_\_0004\_\_：\_\_CODE\_\_0000\_\_ + \_\_CODE\_\_0001\_\_ 支持多游戏 × 多模式 × 多 seed 矩阵实验，自动采集逐步轨迹 JSONL。8 runs 产出 \_\_CODE\_\_0002\_\_ + 8 个轨迹文件 + \_\_CODE\_\_0003\_\_。
- \_\_BOLD\_\_0001\_\_：soft target lock（防 target thrashing）、guide-signature change detection（检测 guide 路径变化）、coin demand override（强制导航到 coin table）已移植到 \_\_CODE\_\_0000\_\_。但 soft lock 在 tap-guide 场景下反而降低了 tap 频率（rule composite 从 0.150 降到 0.10），已修复：tap 后释放 lock。
- \_\_BOLD\_\_0000\_\_：批量实验中 multi-bus 和 multi-bus-memory 均达 \_\_BOLD\_\_0001\_\_（2 seed 一致），确认记忆读回 + 总线通信的组合是当前最佳配置。
- \_\_BOLD\_\_0002\_\_：\_\_CODE\_\_0000\_\_ 已接入 \_\_CODE\_\_0001\_\_，每步写入 JSONL（player/action/keyNumbers/reason），可直接用于后续 VLM 微调。
- \_\_BOLD\_\_0000\_\_：674 passed, 0 failed, ruff 全绿。

## 1.2 1. 分层多 Agent 架构（实验 F）

### 1.2.1 设计

| 层      | 模型                      | 频率               | 延迟   | 职责                    |
|--------|-------------------------|------------------|------|-----------------------|
| L0 执行层 | Rule engine (tap-guide) | 每步               | ~0ms | 跟随当前 plan 执行 move/tap |
| L1 战术层 | 本地 VLM (gemma-4-E4B)    | 每 5 步或 stuck     | ~5s  | 看截图判断当前状态，修正短期目       |
| L2 战略层 | 云端 API (kimi-k2.7-code) | 每 15 步或 phase 切换 | ~3s  | 看 probe state 做长程规划   |

### 1.2.2 实现

- `__CODE_0000__`： `__CODE_0001__` 类， `__CODE_0002__` 按频率调度三层。
- `__CODE_0000__`：注册 `__CODE_0001__` 模式。
- L2 输出 `__CODE_0000__`，存入 `__CODE_0001__`。
- L1 输出 `__CODE_0000__` 或 `__CODE_0001__`，覆盖 L0 动作。

### 1.2.3 批量实验结果

| 模式                         | Mean Composite             | Mean Activity | Mean Latency | L1 calls | L2 calls |
|----------------------------|----------------------------|---------------|--------------|----------|----------|
| <code>__BOLD_0000__</code> | <code>__BOLD_0000__</code> | 1.000         | 21.9s        | —        | —        |
| multi-bus-memory           | 0.215                      | 0.931         | 23.4s        | —        | —        |
| hierarchical               | 0.150                      | 0.000         | 205.1s       | 6        | 2        |
| rule                       | 0.101                      | 0.672         | 34.7s        | —        | —        |

`__BOLD_0000__`：

1. hierarchical 的 activity=0 是因为 L1（本地 VLM）未启动，L2 的 JSON 被 kimi-k2.7-code 的思考链截断，导致 L0 rule engine 收到的 macro\_plan 为空，退化为纯 wait。
2. L2 调用 2 次（step 0 和 step 15），每次 ~3s；L1 调用 6 次（step 0/5/10/15/20/25），每次因连接拒绝而 ~0s 但记录 warning。
3. `__BOLD_0000__`：(a) 本地 VLM 常驻服务；(b) L2 用”只输出 JSON”系统提示或 tool calling；(c) L1 失败时 fallback 到 PIL 视觉分析。

## 1.3 2. Node.js 高级逻辑移植（实验 H）

从同学的 `__CODE_0000__`（2185 行）移植 3 个机制：

| 机制                     | 作用                                       | 实现                            |
|------------------------|------------------------------------------|-------------------------------|
| Soft target lock       | 选定 target 后锁定 8 步，防 target thrashing     | <code>__CODE_0000__</code> ：a |
| Guide-signature change | 检测 guide 路径/角度变化，触发重规划                   | <code>__CODE_0000__</code> ：p |
| Coin demand override   | station 需要 coins 但玩家没有时，强制导航到 coin table | <code>__CODE_0000__</code> ：松 |

`__BOLD_0001__`：soft lock 在 tap-guide 场景下导致 tap 后仍锁定旧 target，新 target 无法被选中，tap 频率从 24/30 降到 19/30。`__BOLD_0002__`：tap 动作后立即释放 lock（`__CODE_0000__`）。

## 1.4 3. 批量实验框架 (实验 G)

### 1.4.1 架构

```
““
src/experiments/batch_runner.py —BatchConfig + run_batch()
src/experiments/analyze_batch.py —读取 batch_results.json → Markdown 表格
src/experiments/exp_batch_matrix.py —预定义矩阵配置 (1 game × 4 modes × 2 seeds)
““
```

### 1.4.2 使用方式

```
““python
from src.experiments.batch_runner import BatchConfig, run_batch

config = BatchConfig(
games={"SSD_00461P01": "/path/to/game.html"},
modes=["rule", "multi-bus", "multi-bus-memory", "hierarchical"],
seeds=[42, 123],
max_steps=30,
collect_dataset=True, # 自动写入轨迹 JSONL
output_dir="batch_results",
)
results = asyncio.run(run_batch(config))
““
```

### 1.4.3 产出

- \_\_CODE\_0000\_\_: 汇总所有 run 的 composite/activity/details
- \_\_CODE\_0000\_\_: 每步 player/action/keyNumbers/reason
- \_\_CODE\_0000\_\_: Markdown 对比表格

### 1.4.4 轨迹数据格式

每行一个 JSON 对象:

```
““json
{"player": {"x": 2.53, "z": 12.78}, "action": "tap", "keyNumbers": {"_failCount": 0},
"reason": "tap_guide_dist=1.95"}
““
```

““

可直接用于：

1. VLM 微调数据（next\_probe\_action / probe\_action\_effect 任务）
2. 离线回放分析（stall 检测、target thrashing 诊断）
3. A/B 对比可视化

## 1.5 4. 同学系统对比分析

对比 \_\_CODE\_0000\_\_ 的 Node.js 系统：

| 维度      | 同学 Node.js                                               | 我们 Python                             |
|---------|----------------------------------------------------------|---------------------------------------|
| 游戏覆盖    | 12 游戏完整驱动                                                | 1 游戏 profile + 22 游戏 HTML 无           |
| 控制循环    | coin override + soft lock + guide signature + A* routing | soft lock + guide sig + coin override |
| 每步延迟    | ~2.9s (Playwright + probe)                               | ~0.8s (rule) / ~22s (multi-bus)       |
| 数据采集    | __CODE_0000__ 写 JSONL                                    | __CODE_0000__ 写 JSONL (本              |
| VLM 训练  | 10,336 样本 7 任务 QLoRA                                     | 15,083 样本 7 任务 (processed-runs        |
| 多 Agent | 无                                                        | 6 角色总线 + 记忆 + Critic                  |

## 1.6 5. 多游戏泛化实验（第四轮）

### 1.6.1 5.1 Generic fallback 让 22 游戏可驱动

\_\_CODE\_0000\_\_ 新增 \_\_CODE\_0001\_\_ (floating joystick + 单位基线，未校准) + \_\_CODE\_0002\_\_。\_\_CODE\_0003\_\_ 对无 profile 游戏不再抛错，改用 generic 驱动（移动方向不可靠但 tap 坐标有效）。

### 1.6.2 5.2 Probe 终止假阳性修复

\_\_BOLD\_0002\_\_: Cocos 引擎标志 \_\_CODE\_0000\_\_ (含“finish”)被 probe WIN 正则误命中，以及 Logo/火堆等常驻 UI 被标“completion-like”——导致 00482/00342 在第一个动作后被误报 \_\_CODE\_0001\_\_，1 步假阳性、composite 虚高 0.700。

\_\_BOLD\_0002\_\_: \_\_CODE\_0000\_\_ 改为佐证制——要求 win/victory 面板节点 / 胜利 analytics / 非 \_\_CODE\_0001\_\_ 管理器的强胜利标志之一，排除 lose/fail (保留 00461 失败重试)。

| 游戏         | 校准? | 模式               | 步数 | composite       | activity | tap | stall |
|------------|-----|------------------|----|-----------------|----------|-----|-------|
| 00461 塔防   | cal | rule             | 25 | 0.106           | 0.71     | 17  | 7     |
| 00461 塔防   | cal | multi-bus-memory | 25 | ___BOLD_0000___ | 1.00     | 24  | 0     |
| 00482 砍树   | GEN | rule             | 25 | 0.150           | 0.00     | 0   | 24    |
| 00482 砍树   | GEN | multi-bus-memory | 25 | 0.150           | 0.00     | 0   | 24    |
| 00736 养蛙捕鱼 | GEN | rule             | 25 | 0.269           | 0.79     | 20  | 5     |
| 00736 养蛙捕鱼 | GEN | multi-bus-memory | 25 | ___BOLD_0000___ | 1.00     | 25  | 0     |
| 00342 建造合并 | GEN | rule             | 25 | 0.150           | 0.00     | 0   | 24    |
| 00342 建造合并 | GEN | multi-bus-memory | 25 | 0.150           | 0.00     | 0   | 24    |
| 00532 瀑布巨木 | GEN | rule             | 25 | 0.150           | 0.00     | 0   | 24    |
| 00532 瀑布巨木 | GEN | multi-bus-memory | 25 | 0.150           | 0.00     | 0   | 24    |

### 1.6.3 5.3 修正后多游戏结果（5 游戏 × 2 模式）

\_\_\_BOLD\_0000\_\_\_:

1. \_\_\_BOLD\_0000\_\_\_——与已校准 00461 持平。记忆读回补偿了未校准基线：记住成功 tap 模式后 activity 0.79→1.00、stall 5→0。
2. \_\_\_BOLD\_0000\_\_\_ 在所有 5 游戏上一致成立。
3. 00482/00342/00532 的 activity=0 是 \_\_\_BOLD\_0000\_\_\_：未校准基线 → 方向全错 → 需该游戏的 profile 校准。
4. 10 个轨迹 JSONL 已采集 (\_\_\_CODE\_0000\_\_\_)，可直接用于 VLM 微调。

### 1.6.4 5.4 全游戏自动校准 (auto\_calibrate.py -all, 22 游戏)

对全部 22 个 \_\_\_CODE\_0000\_\_\_ 游戏跑自动校准（4 方向 joystick 脉冲 + 返回脉冲 + warmup + 重试 + moveByCocosInput 回退），得到 joystick 基线或识别为非 joystick 游戏。

\_\_\_BOLD\_0000\_\_\_:

| 类型                | 数量 | 游戏                                                                                              |
|-------------------|----|-------------------------------------------------------------------------------------------------|
| A 类 (joystick 驱动) | 5  | 00440 清障通车、00483 吸沙抽水、00496 电网抓丧尸、00517 末世旅店、00526 末世旅店                                         |
| A 类 (已有校准)        | 2  | 00461 塔防、00736 捕鱼                                                                               |
| B 类 (tap-to-move) | 15 | 00219、00332、00342、00382、00394、00427、00434、00475、00482、00526、00532、00533、00534、00535、00536、00537 |
| C 类 (probe 失败)    | 0  | —                                                                                               |

\_\_\_BOLD\_0000\_\_\_:

### 1.6.5 5.5 Tap-to-Move 驱动策略

为 B 类 (tap-to-move / 自动移动) 游戏新增 \_\_\_CODE\_0000\_\_\_ 驱动：不走路，直接 tap guide 目标的屏幕坐标 (probe 的 design→CSS 映射，无需 joystick 基线校准)。

| 游戏          | screen_right  | screen_down    |
|-------------|---------------|----------------|
| 00440 清障通车  | (-6.42, 2.34) | (-2.34, -6.42) |
| 00483 吸沙抽水  | (3.41, -3.41) | (2.42, 2.42)   |
| 00496 电网抓丧尸 | (0.75, -0.90) | (0.82, 3.57)   |
| 00517 末世旅店  | (7.42, -0.00) | (0.00, 9.44)   |
| 00522 地下炸矿  | (4.92, 0.00)  | (-0.27, 3.46)  |

15 个 B 类游戏已自动填充 \_\_CODE\_0001\_\_ profile。

\_\_CODE\_0000\_\_ 函数自动分类：A (joystick) / B (tap-only) / C (probe 失败),  
\_\_CODE\_0001\_\_ 自动选择驱动类型。

## 1.7 6. 全游戏 × 多模式批量矩阵

### 1.7.1 6.1 A\_full (7 个 joystick 游戏 × 4 模式 × 2 seeds = 48 runs)

| 游戏          | rule  | multi-bus-memory | multi-bus | hierarchical |
|-------------|-------|------------------|-----------|--------------|
| 00440 清障通车  | 0.184 | 0.156            | 0.156     | 0.150        |
| 00461 塔防    | 0.113 | 0.300            | 0.297     | 0.150        |
| 00483 吸沙抽水  | 0.139 | 0.300            | 0.300     | 0.150        |
| 00496 电网抓丧尸 | 0.275 | 0.150            | 0.150     | 0.150        |
| 00517 末世旅店  | 0.150 | 0.150            | 0.150     | 0.150        |
| 00522 地下炸矿  | 0.215 | 0.240            | 0.300     | —            |

- \_\_BOLD\_0000\_\_ (activity=1.00, stall=0)。
- \_\_BOLD\_0000\_\_——确定性策略比记忆启发式更有效。
- \_\_BOLD\_0000\_\_ (activity=0)：云端 API 的 L2 macro-plan 没有被 L0 规则引擎有效执行。

### 1.7.2 6.2 B\_tap (15 个 tap-only 游戏 × 2 模式 × 2 seeds = 60 runs)

| 结果            | 游戏                                              | 数量            |
|---------------|-------------------------------------------------|---------------|
| __BOLD_0000__ | 00382、00394、00475、00526、00532、00594、00669、00742 | __BOLD_0000__ |
| 0.150         | 00219、00332、00342、00427、00434、00482、00733       | 7             |

\_\_BOLD\_0000\_\_——tap-only 驱动有效：tap guide 目标的屏幕坐标产生真实交互。7/15 为 0.150 (tap 有效但无移动, activity=0)。



| 游戏       | rule  | multi-bus-memory | hierarchical (API) |
|----------|-------|------------------|--------------------|
| 00461 塔防 | 0.106 | 0.056            | 0.150              |
| 00736 捕鱼 | 0.275 | 0.237            | 0.150              |

## 1.8 7. 云端 API 策略生成 vs 纯 rule

云端 API 的 L2 macro-plan 没有被 L0 规则引擎有效执行 (activity=0)。L2 输出需要更强的行动契约。

## 1.9 8. VLM 视觉管线

| 游戏       | probe_only | pil_vision      | vlm_gemma |
|----------|------------|-----------------|-----------|
| 00461 塔防 | 0.044      | ___BOLD_0000___ | 0.150     |
| 00736 捕鱼 | 0.269      | 0.269           | 0.150     |

- \_\_\_BOLD\_0000\_\_\_ (0.044→0.087, tap 7→14, stall 17→10)。
- \_\_\_BOLD\_0000\_\_\_ (activity=0, tap=0) ——本地小模型输出太慢且无法解析为有效动作。

## 1.10 9. 训练数据生成

从 125 条批量实验轨迹 (7 joystick + 15 tap-only 游戏 × 4 模式 × 2 seeds) 离线转换为同事 7 任务训练格式：

| 任务                        | 新增样本            | 合并后总计           |
|---------------------------|-----------------|-----------------|
| probe_action_effect       | +3040           | 5531            |
| information_gain_judgment | +3040           | 6094            |
| progression_grounding     | +3165           | 5806            |
| pulse_response_grounding  | +159            | 1594            |
| failure_recovery          | +9              | 23              |
| ___BOLD_0000___           | ___BOLD_0000___ | ___BOLD_0000___ |

转换器：\_\_\_CODE\_0000\_\_\_，纯离线计算（相邻步 diff → changed\_fields / information\_gain / displacement / stall diagnosis）。

## 1.11 10. L2 输出契约修复

\_\_\_BOLD\_0000\_\_\_：云端 API 的 L2 macro-plan 是抽象文本，L0 无法执行 → hierarchical 全部 activity=0。

\_\_\_BOLD\_0001\_\_\_: L2 prompt 改为要求输出可执行指令列表 (tap/move 带坐标), 存入 \_\_\_CODE\_0000\_\_\_ 动作队列, L0 逐步弹出执行。

\_\_\_BOLD\_0000\_\_\_:

| 模式                      | composite | L2 calls | latency/step |
|-------------------------|-----------|----------|--------------|
| hierarchical (v3, 指令队列) | 0.055     | 1        | 18.1s        |
| rule_baseline           | 0.114     | 0        | 0.84s        |
| multi_bus_memory        | 0.134     | 0        | 0.88s        |

\_\_\_BOLD\_0000\_\_\_: L2 指令队列架构正确 (24/25 步来自 L2 指令), 但 \_\_\_BOLD\_0001\_\_\_ 没有视觉, 坐标全是幻觉。composite 反而更差 (0.055 vs rule 0.114)。

\_\_\_BOLD\_0001\_\_\_: L2 改为输出目标名称 (\_\_\_CODE\_0000\_\_\_), L0 用 probe 的 screenPosition 自行映射坐标。这样 L2 不需要视觉, L0 保留几何准确性。

## 1.12 11. 后续建议

1. \_\_\_BOLD\_0001\_\_\_: 用 probe 的 \_\_\_CODE\_0000\_\_\_ 脉冲自动测量每游戏的 screen→world 基线, 批量生成 profile。
2. \_\_\_BOLD\_0000\_\_\_: systemd 保持 gemma-4-E4B 运行, 让 hierarchical L1 可用。
3. \_\_\_BOLD\_0000\_\_\_: kimi-k2.7-code 加”只输出 JSON”系统提示。
4. \*\_\_\_ITALIC\_0000\_\_\_ routing 移植 \*\*: 从 00864/00867 驱动移植。
5. \_\_\_BOLD\_0001\_\_\_: CI/CD 中跑 \_\_\_CODE\_0000\_\_\_, 每次代码变更自动对比 composite。